Supplementary Material for "MASS: A Complexity-Optimal Solution for Product Kernel Density Visualization"

Yue Zhong
College of Computer Science and Software Engineering, Shenzhen University
Shenzhen, China
2310273008@email.szu.edu.cn

Tsz Nam Chan*
College of Computer Science and Software Engineering, Shenzhen University
Shenzhen, China
edisonchan@szu.edu.cn

Leong Hou U
Department of Computer and Information Science, University of Macau
Macao, China
ryanlhu@um.edu.mo

Dingming Wu
College of Computer Science and Software Engineering, Shenzhen University
Shenzhen, China
dingming@szu.edu.cn

Wei Tu
Ruisheng Wang
School of Architecture and Urban Planning, Shenzhen University
Shenzhen, China
{tuwei,ruiswang}@szu.edu.cn

Joshua Zhexue Huang
College of Computer Science and Software Engineering, Shenzhen University
Shenzhen, China
zx.huang@szu.edu.cn

ACM Reference Format:
Yue Zhong, Tsz Nam Chan, Leong Hou U, Dingming Wu, Wei Tu, Ruisheng Wang, and Joshua Zhexue Huang. 2026. Supplementary Material for "MASS: A Complexity-Optimal Solution for Product Kernel Density Visualization". In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2 (KDD '26), August 09–13, 2026, Jeju Island, Republic of Korea.* ACM, New York, NY, USA, 3 pages. https://doi.org/10.1145/3770855.3817698

1 Extended Visualization Results

Here, we further compare the traditional KDV and the product KDV results in the Lower Nob Hill region of San Francisco using the 311-call location dataset. Observe from Figure 1 that each product KDV (by selecting the correct values of b_x and b_y) can resemble the corresponding traditional KDV, no matter which bandwidth parameter b we adopt.

2 Detailed Proofs of All Lemmas and Theorems

2.1 Proof of Lemma 1

Proof. We first prove that the size of MASS is $(2X-1)\times(2Y-1)$ and then prove that the time complexity for constructing MASS is $O(XY)$.

Note that the search space of any pixel with the v^{th} column, i.e., $\mathbf{q}^{(*v)}$, has two endpoints, which are $\mathbf{q}_x^{(*v)} - b_x$ and $\mathbf{q}_x^{(*v)} + b_x$, in the x-axis (see Figure 4b in the main paper). Since there are X pixels in each row, we conclude that there are $2X$ endpoints along the x-axis, indicating that there are $2X-1$ entries (intervals) along the x-axis in MASS (see Figure 4c in the main paper). Based on the similar idea, we can also show that there are $2Y-1$ entries along the y-axis in MASS.

Since MASS does not have the same size for each entry (see Figure 4c in the main paper), we need to store the positions of the endpoints in the x-axis and the y-axis for each entry in order to correctly identify it (e.g., we need to store $\mathbf{q}_x^{(*1)} + b_x$, $\mathbf{q}_x^{(*3)} - b_x$, $\mathbf{q}_y^{(2*)} - b_y$, and $\mathbf{q}_y^{(1*)} + b_y$ in the $(3,2)$-entry of MASS in Figure 4c in the main paper). To construct MASS with this goal, we need to first maintain the sorted order for those coordinates of the endpoints in the x-axis (the y-axis). Then, we can build this $(2X-1)\times(2Y-1)$ matrix. Note that we have the following sorted lists in advance (due to the regular property of pixel)

$$\mathbf{q}_x^{(*1)} - b_x \le \mathbf{q}_x^{(*2)} - b_x \le \ldots \le \mathbf{q}_x^{(*X)} - b_x$$
$$\mathbf{q}_x^{(*1)} + b_x \le \mathbf{q}_x^{(*2)} + b_x \le \ldots \le \mathbf{q}_x^{(*X)} + b_x$$

By merging these two lists with $O(X)$ time, we can obtain the sorted order for those endpoints in the x-axis (see Figure 4c in the main paper). Similarly, we can also obtain the sorted order for those endpoints in the y-axis in $O(Y)$ time. In addition, building this matrix takes $O(XY)$ time. Hence, we conclude that the time complexity of constructing MASS is $O(XY)$. □

2.2 Proof of Lemma 2

Proof. Note that the initialization step (i.e., setting the polynomial terms to be 0 for each entry e) needs to access $(2X-1)\times(2Y-1)$ entries in MASS (see Lemma 1 in the main paper). Therefore, this step takes $O(XY)$ time. In addition, using the binary search method along the x-axis and the y-axis for all data points takes $O(n\log X + n\log Y) = O(n\log XY)$ time. Furthermore, updating all entries e with respect to all data points takes $O(n)$ time. Hence, we conclude that the time complexity is $O(XY + n\log XY)$. □

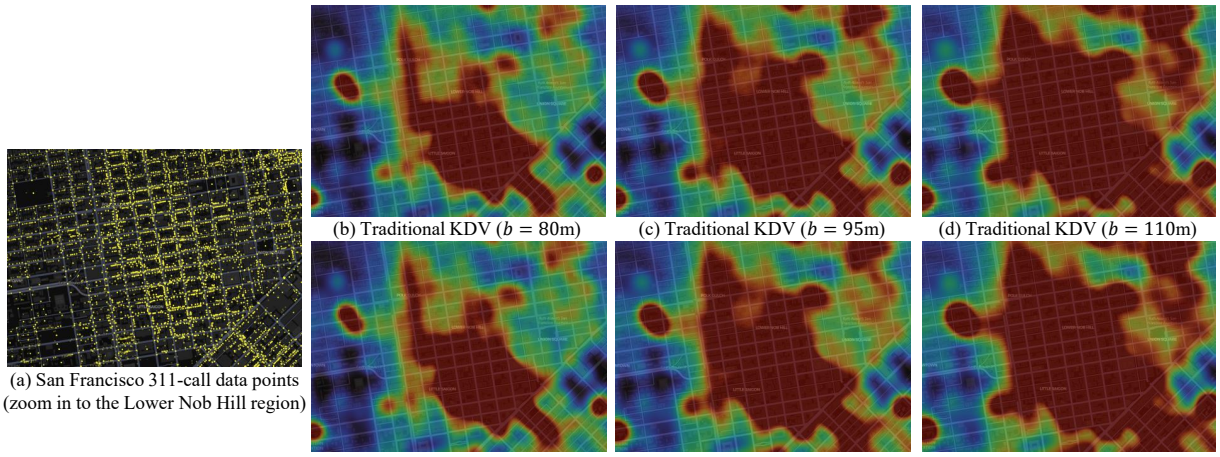
2.3 Proof of Lemma 3

Proof. Recall that MASS with augmented polynomial terms $T_{P(e)}^{(i,j)}$ contains $(2X-1)\times(2Y-1)$ entries (see Lemma 1 in the main paper). Therefore, based on [1], constructing the prefix matrices (with respect to all (i,j)-pairs) takes $O(XY)$ time. To evaluate $T_{\mathbb{S}(\mathbf{q})}^{(i,j)}$ with the search space $\mathbb{S}(\mathbf{q})$, we need to utilize the binary search

*Corresponding author.

KDD 126, Jeju Island, Republic of Korea

ACM ISBN 979-8-4007-2259-2/2026/08
https://doi.org/10.1145/3770855.3817698



(a) San Francisco 311-call data points (zoom in to the Lower Nob Hill region)

(b) Traditional KDV ($b = 80$m)

(c) Traditional KDV ($b = 95$m)

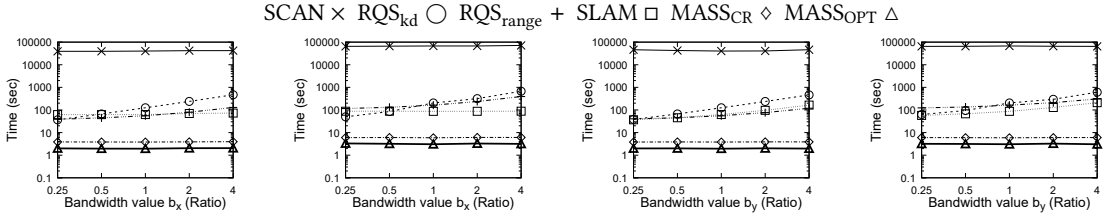
(d) Traditional KDV ($b = 110$m)

(e) Product KDV ($b_x = 73$m and $b_y = 73$m)

(f) Product KDV ($b_x = 88$m and $b_y = 88$m)

(g) Product KDV ($b_x = 103$m and $b_y = 103$m)

Figure 1: Generating traditional and product KDVs in the Lower Nob Hill region for the San Francisco 311-call location dataset.



(a) London (varying b_x) (b) San Francisco (varying b_x) (c) London (varying b_y) (d) San Francisco (varying b_y)

Figure 2: Response times for computing the product KDV in the London (a and c) and San Francisco (b and d) datasets, varying the bandwidth parameters, b_x (a and b) and b_y (c and d).

method along the x-axis and y-axis for identifying (at most four) correct entries in the corresponding prefix matrix, which takes $O(\log X + \log Y) = O(\log XY)$ time for each pixel. Given that there are $X \times Y$ pixels, this step takes $O(XY \log XY)$ time. Hence, we conclude that the time complexity of computing $T^{(i,j)}_{\mathbb{S}(\mathbf{q})}$ for all pixels $\mathbf{q}$ and (i, j)-pairs is $O(XY \log XY)$. □

2.4 Proof of Theorem 1

Proof. Recall from Lemma 1 and Lemma 2 that the time complexities for constructing MASS $\mathbb{M}$ (line 3) and augmenting $\mathbb{M}$ (line 4) are $O(XY)$ and $O(XY + n \log XY)$, respectively. Furthermore, based on Lemma 3, constructing the prefix matrices (line 5) and computing $T^{(i,j)}_{\mathbb{S}(\mathbf{q})}$ with all (i, j)-pairs for every pixel $\mathbf{q}$ (line 7 throughout the loop in line 6) takes $O(XY \log XY)$ time. In addition, once we have $T^{(i,j)}_{\mathbb{S}(\mathbf{q})}$, we can compute the product kernel density function $\mathcal{D}_P(\mathbf{q})$ (line 8) based on Equation 5 (in the main paper) in $O(1)$ time. Therefore, the time complexity of line 8 throughout the loop in line 6 is $O(XY)$ time. Hence, we can conclude that the time complexity of MASS$_{CR}$ is $O((XY + n) \log XY)$. □

2.5 Proof of Theorem 2

Proof. Since MASS$_{CR}$ needs to access all data points in P (with size n) and all pixels in the plane $\mathbb{P}$ (with size $X \times Y$), the space complexity is at least $O(XY + n)$. Since this solution only needs to maintain the index structure $\mathbb{M}$ (with size $(2X - 1) \times (2Y - 1)$) with all augmented polynomial terms $T^{(i,j)}_{P(e)}$ (see line 3 and line 4 in Algorithm 1 in the main paper) and construct the prefix matrices (with the same size $(2X - 1) \times (2Y - 1)$) for $\mathbb{M}$ (see line 5), the additional space complexity is $O(XY)$. Hence, the space complexity of MASS$_{CR}$ is $O(XY + n)$. □

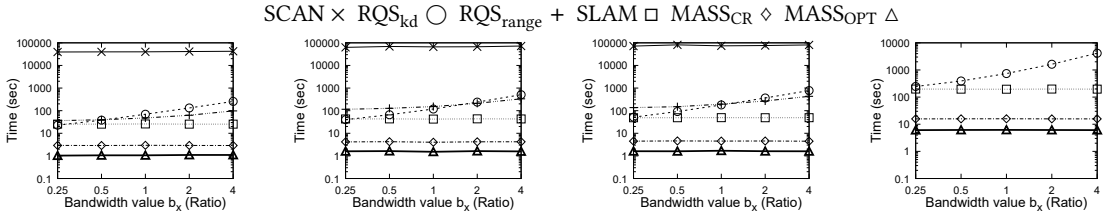
3 Additional Experiments

3.1 Additional Time Efficiency Experiments of All Methods

We follow the same settings in Section 5.2 (in the main paper) to conduct these two experiments to test the efficiency of all methods for the London and San Francisco datasets.

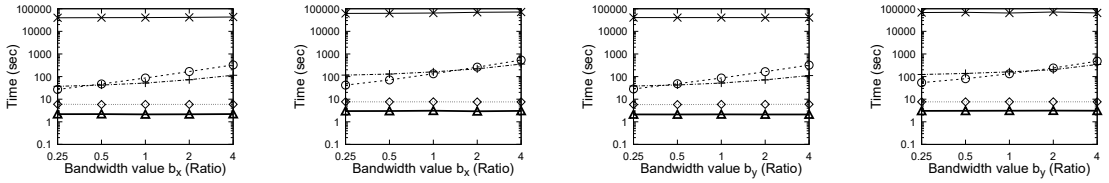
Varying the bandwidth parameter b_x. In this experiment, we test the time efficiency of all methods with respect to different bandwidth parameters b_x. Figures 2a and b show the results of all methods. Like the results in Figures 10a and b (in the main paper), the response times of RQS$_{kd}$ and RQS$_{range}$ are proportional to this bandwidth parameter b_x. The main reason is that a larger bandwidth value b_x leads to a larger search space (see Figure 2b in the main paper), which results in processing more data points for these methods. Unlike RQS$_{kd}$ and RQS$_{range}$, the response times of SCAN, SLAM, MASS$_{CR}$, and MASS$_{OPT}$ are not sensitive to b_x. With the lower time complexities of our methods, MASS$_{CR}$ and MASS$_{OPT}$, we can note that MASS$_{CR}$ can achieve 8.33x to 18.42x speedups over the existing methods. In addition, MASS$_{OPT}$ can further achieve 1.8x to 1.96x speedups compared with MASS$_{CR}$.

Varying the bandwidth parameter b_y. Here, we proceed to investigate the time efficiency of all methods with respect to different



(a) London (varying b_x) (b) San Francisco (varying b_x) (c) London (varying b_y) (d) San Francisco (varying b_y)

Figure 3: Response times for computing the product KDV with the uniform kernel function in the London and San Francisco datasets, varying the bandwidth parameters, b_x (a and b) and b_y (c and d).



(a) London (varying b_x) (b) San Francisco (varying b_x) (c) London (varying b_y) (d) San Francisco (varying b_y)

Figure 4: Response times for computing the product KDV with the triangular kernel function in the London and San Francisco datasets, varying the bandwidth parameters, b_x (a and b) and b_y (c and d).

bandwidth parameters b_y. Figures 2c and d show the results of all methods. Unlike Figures 2a and b, the response time of SLAM is sensitive to b_y since a large b_y results in a large envelope (see Figure 3a in the main paper), indicating that this method needs to process more data points. Observe that the trends of response times of other methods are similar to the ones of varying b_x (based on the similar reasons in Figures 2a and b). Note that our best method, MASS$_{OPT}$, achieves 9.54x to 69.17x speedups compared with the existing methods.

3.2 Other kernels

In this section, we proceed to investigate the time efficiency of computing the product KDV with other kernels, including the uniform kernel and the triangular kernel (see Table 2 in the main paper) on the London and San Francisco datasets. To conduct the experiments, we follow the same settings in Section 5.2 (in the main paper) for varying the bandwidth parameter b_x and the bandwidth parameter b_y for testing the time efficiency.

Uniform kernel. Figures 3a and b show the results of all methods by varying b_x. Note that we can seamlessly reuse all methods (without significant modifications and processing overheads) to support the uniform kernel function. The trends of response times in Figures 3a and b are similar to the ones of Figures 2a and b, and Figures 10a and b (in the main paper). However, since MASS$_{CR}$ and MASS$_{OPT}$ only need to maintain a single term $T^{(0,0)}_{P(e)}$ (based on Equation 10 in the main paper) for each entry e of the MASS structure (instead of nine terms for the Epanechnikov kernel in Figure 5 in the main paper), the response times of these two methods are smaller than the ones of the Epanechnikov kernel. With the lower time complexities of MASS$_{CR}$ and MASS$_{OPT}$, we can observe that MASS$_{CR}$ can achieve 2.78x to 8.81x speedups compared with all existing methods. Moreover, MASS$_{OPT}$ further achieves 2.62x to 3.39x speedups compared with MASS$_{CR}$.

Figures 3c and d further show the results of all methods by varying b_y. Based on the similar reasons for the experiment of varying b_x, we can observe that (1) the trends of response times in Figures 3c and d are also similar to Figures 2c and d, and Figures 10c and d (in the main paper), and (2) MASS$_{CR}$ and MASS$_{OPT}$ can achieve 2.71x to 26.22x speedups compared with all existing methods.

Triangular kernel. Figures 4a and b show the results of all methods by varying b_x. Since SLAM cannot support the triangular kernel, we do not include this method for comparison. Furthermore, the space consumption of RQS$_{range}$ is more than 16GB in the Manhattan dataset. Therefore, the results of this method are also omitted in this dataset. Like the results of the Epanechnikov kernel and the uniform kernel, the trends of response times in Figures 4a and b are similar to the ones of Figures 2a and b (Epanechnikov kernel), Figures 10a and b in the main paper (Epanechnikov kernel), and the ones of Figures 3a and b (uniform kernel). Compared with the uniform kernel, MASS$_{CR}$ and MASS$_{OPT}$ need to maintain a larger MASS structure and access more regions (see Figure 15 in the main paper). Therefore, these two methods are slower than the ones of the uniform kernel. Despite this, MASS$_{CR}$ and MASS$_{OPT}$ can still achieve 2.71x to 74.65x speedups compared with the existing methods (due to the lower time complexities of these two methods).

Figures 4c and d further show the response times of all methods by varying b_y. Based on the similar reasons in the experiment of varying b_x, we note that the trends of response times in Figures 4c and d are similar to the ones of Figures 2c and d (Epanechnikov kernel), Figures 10c and d in the main paper (Epanechnikov kernel), and Figures 3c and d (uniform kernel). In addition, MASS$_{CR}$ and MASS$_{OPT}$ can still achieve 4.9x to 110.82x speedups compared with the existing methods.

References

[1] Ching-Tien Ho, Rakesh Agrawal, Nimrod Megiddo, and Ramakrishnan Srikant. 1997. Range Queries in OLAP Data Cubes. In *SIGMOD*. ACM, 73–88. doi:10.1145/253260.253274